

Elenco dei requisiti funzionali (versione rivista e completata)

Marco

August 6, 2026

Contents

1	Perche' esiste questo file	3
2	Come si legge l'elenco	4
3	Cosa e' stato corretto rispetto alla lista vecchia	4
3.1	Errori veri e propri	5
3.2	Requisiti che mancavano del tutto	5
3.3	Requisiti che erano ottimisti	6
4	RF01 - Casi d'uso dell'applicazione web	6
4.1	RF01.1 - Visualizzazione dei dati in tempo reale e corretti . .	6
4.1.1	RF01.1.1 - Mappa del traffico	6
4.1.2	RF01.1.2 - Elenco dei treni con posizione, stato e ritardo	6
4.1.3	RF01.1.3 - Elenco delle stazioni con stato operativo e treni presenti	7
4.1.4	RF01.1.4 - Aggiornamento continuo senza ricaricare la pagina	7
4.1.5	RF01.1.5 - Indicatori di sintesi	8
4.2	RF01.2 - Visualizzazione dei guasti	8
4.2.1	RF01.2.1 - Elenco degli allarmi aperti	8
4.2.2	RF01.2.2 - Distinzione fra guasto segnalato e guasto dedotto	8
4.3	RF01.3 - Modifica dei dati riservata all'amministratore	9
4.3.1	RF01.3.1 - Stazioni	9
4.3.2	RF01.3.2 - Treni	9
4.3.3	RF01.3.3 - Tratte elementari (il segmento fra due stazioni)	9

4.3.4	RF01.3.4 - Itinerari e assegnazione dei convogli	9
4.3.5	RF01.3.5 - Integrita' referenziale sulle cancellazioni . . .	9
4.4	RF01.4 - Comandi operativi	10
4.4.1	RF01.4.1 - Invio della squadra di manutenzione	10
4.4.2	RF01.4.2 - Presa in carico e chiusura di un allarme . . .	10
4.4.3	RF01.4.3 - Soppressione di un convoglio fermo in stazione	10
4.5	RF01.5 - Consultazione dello storico	11
4.6	RF01.6 - Accesso al sistema	11
5	RF02 - Correttezza del sistema gestionale	11
5.1	RF02.1 - Gestione dei guasti	12
5.1.1	RF02.1.1 - Guasto della stazione	12
5.1.2	RF02.1.2 - Guasto del treno	13
5.1.3	RF02.1.3 - Ciclo di vita del guasto	14
5.1.4	RF02.1.4 - Chiusura automatica quando la causa rientra	14
5.2	RF02.2 - Gestione dei treni	14
5.2.1	RF02.2.1 - Gestione dei ritardi	14
5.2.2	RF02.2.2 - Servire le informazioni di stato del convoglio	15
5.2.3	RF02.2.3 - Andata e ritorno sullo stesso itinerario . . .	16
5.2.4	RF02.2.4 - Passaggi asimmetrici in stazione	16
5.3	RF02.3 - Gestione delle stazioni	16
5.3.1	RF02.3.1 - Informazioni rese alla visualizzazione	16
5.3.2	RF02.3.2 - Heartbeat verso la centrale	17
5.3.3	RF02.3.3 - Keepalive dei sensori verso la stazione . . .	17
5.3.4	RF02.3.4 - Stato operativo dichiarato dalla stazione . .	18
5.4	RF02.4 - Validazione del nodo di campo	18
5.4.1	RF02.4.1 - L'ID dichiarato viene validato sul database centrale	18
5.4.2	RF02.4.2 - Esito negativo: il processo non prosegue . .	18
5.4.3	RF02.4.3 - Dipendenza fattorizzata	18
5.5	RF02.5 - Modifica della tratta a sistema acceso	19
5.5.1	RF02.5.1 - Treno fuori dalla nuova tratta: viene fer- mato in attesa di soppressione	19
5.5.2	RF02.5.2 - Treno dentro la nuova tratta: la segue dal punto in cui si trova	19
5.6	RF02.6 - Rilevazione automatica delle anomalie (watchdog) .	20
5.6.1	RF02.6.1 - Stazione muta oltre la soglia => OFFLINE e guasto automatico	20
5.6.2	RF02.6.2 - Treno fermo fuori stazione oltre la soglia => allarme automatico	20

5.6.3	RF02.6.3 - Snapshot periodico verso la dashboard . . .	21
5.7	RF02.7 - Storicizzazione	21
6	RF03 - Ecosistema di agenti e comunicazione	21
6.1	RF03.1 - Un processo per convoglio e un processo per stazione	22
6.2	RF03.2 - MQTT dal campo alla centrale	22
6.3	RF03.3 - Un solo passaggio fisico, due strade indipendenti . .	22
6.4	RF03.4 - REST dai sensori al nodo di campo	23
6.5	RF03.5 - Itinerario e prossima stazione serviti dalla centrale .	23
7	RF04 - Sicurezza degli accessi e dei canali	24
7.1	RF04.1 - Autenticazione tradizionale	24
7.2	RF04.2 - Autorizzazione per ruolo	24
7.3	RF04.3 - Canali cifrati	25
8	Grafo delle dipendenze aggiornato	25
9	Matrice di tracciabilita'	26
10	Riepilogo di cio' che e' dichiarato non fatto	27
11	Nota sui file vicini	28

1 Perche' esiste questo file

Nella documentazione finale (`doc/documentazioneFinale/DocumentazioneDelSistemaMonitoraggioE` sezione "Elenco maggiormente formale dei requisiti") l'elenco degli RF era stato buttato giu' di getto all'inizio del progetto: due soli requisiti radice, RF01 (casi d'uso) e RF02 (correttezza del sistema gestionale), con sotto i rami del dominio.

Quella lista adesso non regge piu' per tre motivi:

1. **e' rimasta indietro rispetto al codice:** nel frattempo sono stati implementati heartbeat, watchdog, storicizzazione, andata/ritorno, autenticazione e TLS, e di tutto questo nella lista non c'e' traccia;
2. **ha due voci sbagliate**, non solo incomplete: lo storico dichiarato "non richiesto" (il PDF del prof lo chiede esplicitamente, ed e' pure implementato) e la frase sulla modifica della tratta che dice il contrario di quello che intendeva;

3. **non distingue cio' che e' fatto da cio' che non lo e'**, e all'interrogazione questa e' proprio la distinzione che conviene avere chiara: meglio dire "questo requisito l'ho scritto e l'ho implementato a meta', ecco dove si rompe" che farsi trovare impreparati.

Questo file quindi **sostituisce** quella lista. La documentazione finale puo' rimandare qui.

2 Come si legge l'elenco

Ogni requisito e' scritto con quattro informazioni:

enunciato cosa il sistema deve fare, in una frase, al presente;

origine da dove viene il requisito. Tre possibilita':

- **PDF** = chiesto dal testo del prof (`doc/Progetto finale 2025_26.pdf`),
- **derivato** = non scritto nel PDF ma necessario perche' un requisito PDF stia in piedi,
- **scelta** = decisione progettuale mia, il PDF lasciava liberta';

stato IMPLEMENTATO, PARZIALE (con la spiegazione di cosa manca), FUORI AMBITO (deciso di non farlo, e si dice perche');

dove il punto del codice che lo realizza e la macchina a stati corrispondente nei diagrammi (**T*** treno, **S*** stazione, **M*** centrale), cosi' il requisito e' navigabile fino alla riga.

La numerazione e' gerarchica e la gerarchia ha un significato preciso, lo stesso del grafo delle dipendenze: **RFOX.Y** e' una **parte** di **RFOX** (decomposizione), mentre "dipende da" e' un'altra relazione e viene scritta a parte. Sono due cose diverse e nel grafo hanno due tipi di freccia diversi.

3 Cosa e' stato corretto rispetto alla lista vecchia

Riassunto delle modifiche, cosi' e' evidente cosa e' cambiato e non bisogna fare il diff a mano.

3.1 Errori veri e propri

#	dove	com'era
1	vecchio RF01 punto 4	“costruzione di uno storico (funzionalità non richiesta)”
2	vecchio RF02 modifica tratta	“il treno non e' fuori dalla tratta il treno viene fermato in attesa
3	vecchio RF02 guasto treno	“se il treno si guasta nella tratta si vuole che la stazione tratta’
4	scenari, Centrale	“interrogare il database ogni 10 secondi per ogni tipo di visualizz

3.2 Requisiti che mancavano del tutto

Erano tutti già implementati, semplicemente non erano mai stati messi per iscritto:

- heartbeat stazione -> centrale e keepalive sensori -> stazione (RF02.3.2, RF02.3.3), che il PDF chiede in modo esplicito nel paragrafo “Guasto del Sensore/Stazione (Fail-Stop)”;
- rilevazione automatica delle anomalie lato centrale, cioè il watchdog (RF02.6): stazione muta e treno fermo troppo a lungo fra due stazioni. Il secondo e' l'esempio testuale del PDF sotto “Alerting”;
- viaggio di andata e ritorno con inversione al capolinea (RF02.2.3), chiesto dal PDF (“sia all'andata, che al ritorno”);
- passaggi asimmetrici: il treno esce dalla stazione di partenza senza esserci entrato e entra al capolinea senza uscirne (RF02.2.4), altro caso testuale del PDF;
- comandi operativi del tecnico e dell'amministratore (RF01.4): invio della squadra di manutenzione, presa in carico dell'allarme, soppressione del treno fermo in stazione;
- autenticazione e permessi per ruolo (RF01.6 e RF04.1, RF04.2): il PDF chiede due tipologie di utenti con poteri diversi, e la variante 2 permette il login tradizionale al posto di OAuth2;
- cifratura dei canali (RF04.3): la variante 6 costa 5 punti se **non** si fa il TLS, quindi il fatto di averlo fatto e' un requisito da dichiarare;
- ecosistema di agenti, cioè un processo per convoglio e uno per stazione (RF03.1): e' la condizione che il PDF mette in maiuscolo per la sufficienza, e la variante 7 (accorpare le stazioni in un servizio solo) costa altri 5 punti;

- storicizzazione dei transiti e dei cambi di stato (RF02.7), che e' il lato dati di RF01.5.

3.3 Requisiti che erano ottimisti

- RF02.5.2 (dopo la modifica della tratta il treno riprende dal punto in cui si trova) era dato per fatto: in realta' e' **PARZIALE**, il digital twin ricarica l'itinerario ma riparte dalla prima stazione. Sotto e' spiegato dove si rompe.

4 RF01 - Casi d'uso dell'applicazione web

Enunciato: il sistema espone a tecnico e amministratore, attraverso un'unica applicazione web, la situazione corrente della rete e i comandi previsti per il loro ruolo.

Origine: PDF (§3.C, “questo server deve essere accessibile da due tipologie di utenti”).

Nota di lettura: RF01 non aggiunge logica di dominio, la **espone**. Per questo ogni suo ramo scarica la dipendenza su RF02: se il dominio sotto e' sbagliato, la schermata mostra bene un dato sbagliato.

4.1 RF01.1 - Visualizzazione dei dati in tempo reale e corretti

4.1.1 RF01.1.1 - Mappa del traffico

- enunciato: la posizione dei convogli e' visibile su una mappa geografica, con la tratta percorsa e lo stato del convoglio.
- origine: PDF (visione d'insieme del traffico).
- stato: **IMPLEMENTATO**.
- dove: `ClientWebAppIntefacciaUtente/src/pages/TrafficMapPage.tsx`, alimentata da M3 (telemetria) e M11 (websocket).

4.1.2 RF01.1.2 - Elenco dei treni con posizione, stato e ritardo

- enunciato: per ogni convoglio si vedono numero di convoglio, stazione o tratta su cui si trova, stato (FERMO / $IN_{VIAGGIO}$ / EMERGENZA / SOPPRESSO), ritardo accumulato e passeggeri a bordo.

- origine: PDF (il tecnico “visualizza quali tratte segue un certo treno”).
- stato: IMPLEMENTATO.
- dove: GET /api/treni, TrainsPage.tsx; il dato nasce da T8 e viene ingerito da M3.

4.1.3 RF01.1.3 - Elenco delle stazioni con stato operativo e treni presenti

- enunciato: per ogni stazione si vedono lo stato (ONLINE / OFFLINE / GUASTA / MANUTENZIONE), i treni attualmente fermi sui binari e quelli attesi in ingresso e in uscita.
- origine: PDF (il tecnico “visualizza quali treni ci si aspetta in ingresso e in uscita in una certa stazione”).
- stato: IMPLEMENTATO.
- dove: GET /api/stazioni, StationsPage.tsx; stato di stazione = M12, presenze = S11.

4.1.4 RF01.1.4 - Aggiornamento continuo senza ricaricare la pagina

- enunciato: le schermate si aggiornano da sole mentre l’operatore le guarda.
- origine: PDF (obiettivo “bassa latenza”: aggiornare la centrale in tempo reale).
- stato: IMPLEMENTATO.
- dove: WebSocket /ws/realtime (RealtimeWebSocket, M11) lato server, websocketClient.ts + useRealtimeUpdates.ts lato client, con riconnessione automatica ogni 5s.
- **correzione:** la vecchia frase “interrogare il database ogni 10 secondi per ogni tipo di visualizzazione” descriveva un polling che non e’ quello che fa il sistema. Il meccanismo e’ **push**: ogni evento rilevante (telemetria, transito, guasto, comando) viene spinto sul canale nel momento in cui accade; i 10 secondi sono il periodo dello snapshot di riallineamento (FaultMonitor.broadcastSnapshot), che serve a chi si e’ appena collegato o ha perso messaggi, non a fare da orologio dell’interfaccia.

4.1.5 RF01.1.5 - Indicatori di sintesi

- enunciato: una dashboard riassume treni in circolazione, stazioni operative, guasti aperti e ritardo medio.
- origine: scelta (il PDF chiede una “visione d’insieme”, la forma e’ mia).
- stato: IMPLEMENTATO.
- dove: GET /api/dashboard -> TrafficLogicEngine.kpiDashboard(), DashboardPage.tsx.

4.2 RF01.2 - Visualizzazione dei guasti

4.2.1 RF01.2.1 - Elenco degli allarmi aperti

- enunciato: si vedono i guasti attivi con sorgente (quale stazione o quale treno), tipo, severita’ e istante di apertura.
- origine: PDF (§ Alerting).
- stato: IMPLEMENTATO.
- dove: GET /api/allarmi, AlertsPage.tsx; il ciclo di vita e’ M7.

4.2.2 RF01.2.2 - Distinzione fra guasto segnalato e guasto dedotto

- enunciato: si distingue il guasto segnalato dal campo (un sensore lo ha dichiarato) da quello dedotto dalla centrale (il nodo ha smesso di parlare).
- origine: derivato da RF02.6: sono due guasti con cause diverse e vanno letti diversamente, perche’ il secondo puo’ anche essere solo un problema di rete.
- stato: IMPLEMENTATO.
- dove: campo sorgenteTipo / creazione via FaultMonitor.creaGuastoAutomatico contro IngestionService.onAlert (M6 e M8).

4.3 RF01.3 - Modifica dei dati riservata all'amministratore

Enunciato del ramo: la CRUD sull'anagrafica della rete e' permessa al solo amministratore; il tecnico vede tutto ma non modifica nulla.

Origine: PDF (§3.C: l'amministratore "puo' aggiungere ed eliminare tratte, puo' modificare le stazioni attraversate dai treni").

4.3.1 RF01.3.1 - Stazioni

- stato: IMPLEMENTATO — POST/PUT/DELETE /api/stazioni[/id], StationEditorModal.tsx.

4.3.2 RF01.3.2 - Treni

- stato: IMPLEMENTATO — POST/PUT/DELETE /api/treni[/id], TrainEditorModal.tsx.

4.3.3 RF01.3.3 - Tratte elementari (il segmento fra due stazioni)

- stato: IMPLEMENTATO — POST/PUT/DELETE /api/tratte-elementari[/id], TrackSegmentEditorModal.tsx.

4.3.4 RF01.3.4 - Itinerari e assegnazione dei convogli

- enunciato: l'amministratore compone l'itinerario come sequenza ordinata di stazioni con i tempi di percorrenza, e ci assegna sopra i convogli.
- stato: IMPLEMENTATO — POST/PUT/DELETE /api/tratte[/id], RouteEditorModal.tsx.
- dipende da: RF02.5 (la modifica deve restare corretta anche a treni gia' in marcia).

4.3.5 RF01.3.5 - Integrita' referenziale sulle cancellazioni

- enunciato: cancellare un'entita' non deve lasciare in giro righe orfane, e non deve essere possibile se qualcosa la sta ancora usando.
- origine: derivato (senza questo la "correttezza" di RF02 dura fino alla prima DELETE).
- stato: IMPLEMENTATO — es. DELETE /api/tratte-elementari/{id} rifiuta se esistono transiti storici su quella tratta; DELETE /api/stazioni/{id} ripulisce prima transiti e storici collegati.

4.4 RF01.4 - Comandi operativi

Nota: questo ramo mancava del tutto nella vecchia lista, dove i comandi comparivano solo come accenno dentro “gestione guasti” della sezione scenari.

4.4.1 RF01.4.1 - Invio della squadra di manutenzione

- enunciato: il tecnico manda gli operatori su una stazione guasta; la stazione passa in MANUTENZIONE e i nodi di campo ne vengono avvisati.
- origine: PDF (§3.A: “un tecnico possa inviare operatori per risolvere immediatamente il guasto”; §3.C: il tecnico “ha il compito di mandare operatori per riparare una stazione”).
- stato: IMPLEMENTATO.
- dove: POST /api/stazioni/{id}/manutenzione -> alert MAINTENANCE_DISPATCHED su railway/alerts; M10 in uscita, S9 in ricezione.

4.4.2 RF01.4.2 - Presa in carico e chiusura di un allarme

- enunciato: il tecnico dichiara risolto un guasto; il guasto viene chiuso, storicizzato e il campo riceve il RESOLVED che sblocca i treni fermi.
- origine: PDF (la centrale “potra’ attivare la procedura di ripristino”).
- stato: IMPLEMENTATO — POST /api/allarmi/{id}/risolvi, M7 + M10; lato treno e’ T9 che aggiorna T11.

4.4.3 RF01.4.3 - Soppressione di un convoglio fermo in stazione

- enunciato: si sopprime una corsa; il convoglio passa in SOPPRESSO e smette di viaggiare.
- origine: PDF (§3.C: l’amministratore “(eventualmente) puo’ sopprimere treni fermi in stazione”).
- stato: IMPLEMENTATO — POST /api/treni/{id}/sopprimi -> alert STOP mirato sul convoglio.
- **nota onesta:** nel PDF la soppressione e’ attribuita all’amministratore, nel filtro di autorizzazione l’ho messa fra i comandi operativi permessi anche al tecnico (FiltroAutorizzazione.richiedeAmministratore,

che lascia passare `/sopprimi`, `/risolvi` e `/manutenzione` a entrambi i ruoli). E' una scelta discutibile: l'ho fatta perche' i tre comandi sono la reazione a un'emergenza e il tecnico e' quello che sta davanti allo schermo quando succede. Se all'interrogazione la si considera una deviazione dal testo, si stringe cambiando una riga in quel metodo.

4.5 RF01.5 - Consultazione dello storico

- enunciato: sono consultabili i transiti passati, con treno, stazione, tratta, verso e istante.
- origine: PDF (§3.C, Storage: “gestire lo storico dei transiti”).
- stato: IMPLEMENTATO.
- dove: `GET /api/transiti` legge `StoricoTransito`; il dato viene scritto da M5.
- **correzione:** nella lista vecchia questa voce era annotata “funzionalita' non richiesta”. E' sbagliato due volte: il PDF la chiede in modo esplicito, e nel frattempo e' stata implementata. Il lato dati (piu' ampio di quello esposto) e' RF02.7.

4.6 RF01.6 - Accesso al sistema

- enunciato: l'accesso alle funzionalita' richiede login; la sessione accompagna ogni chiamata successiva e si chiude con il logout.
- origine: PDF + variante 2 (utenti memorizzati sul server e autenticazione tradizionale al posto di OAuth2, nessuna penalita').
- stato: IMPLEMENTATO — `POST /api/auth/login` e `/logout` (`AuthController`), token di sessione in `SessioniAttive`, `LoginPage.tsx` + `authStore.ts`. Macchina M9.
- rimando: i dettagli di sicurezza stanno in RF04.

5 RF02 - Correttezza del sistema gestionale

Enunciato: lo stato che il sistema mostra e persiste corrisponde a quello che sta realmente succedendo nella rete simulata, anche in presenza di guasti, disconnessioni e modifiche fatte a sistema acceso.

Origine: scelta come formulazione, ma raccoglie gli obiettivi del PDF (affidabilità: “i dati di transito non vadano persi”).

5.1 RF02.1 - Gestione dei guasti

5.1.1 RF02.1.1 - Guasto della stazione

1. RF02.1.1.1 - Guasto temporaneo: perdita della connessione con la centrale

(a) RF02.1.1.1.1 - Cache locale e reinvio in massa

- enunciato: mentre il collegamento e' giu', la stazione accumula in locale tutto cio' che produce e lo rispedisce quando la connessione torna.
- origine: PDF (§3.A: “la stazione deve implementare un meccanismo di caching locale per reinviare i dati una volta ripristinata la connessione”).
- stato: IMPLEMENTATO.
- dove: `Stazioni/.../LocalBuffer.java` + S4 (connettività e store-and-forward), svuotamento guidato da S7. Per la demo la caduta si simula con `POST /stazione/rete/offline` e `/rete/online`.

(b) RF02.1.1.1.2 - Ordine di consegna preservato

- enunciato: il reinvio rispetta l'ordine di generazione; se una singola consegna fallisce, l'evento torna **in testa** alla coda e non in fondo.
- origine: **derivato** (senza questo l'ordine entrata/uscita di uno stesso treno si puo' invertire e lo storico dei transiti diventa incoerente: e' esattamente il dato che RF01.5 mostra).
- stato: IMPLEMENTATO — S5, ciclo di vita dell'evento di stazione.

2. RF02.1.1.2 - Guasto permanente: la stazione non e' percorribile

(a) RF02.1.1.2.1 - Il treno diretto alla stazione guasta si ferma

- enunciato: se la prossima stazione dell'itinerario e' guasta, il convoglio non ci entra e resta fermo dov'e'.
- origine: PDF (§3.A: “la stazione segnala il guasto a ogni treno che entra in stazione, il quale non riparte fino a quando il guasto non e' ripristinato”).

- stato: **IMPLEMENTATO** — T9 riceve il **GUASTO**, T11 tiene l'elenco delle stazioni guaste note al convoglio, T5 usa quell'elenco per trattenere la partenza.
- (b) RF02.1.1.2.2 - Riparte solo al ripristino
- enunciato: il convoglio riparte quando arriva il **RESOLVED** di quella stazione, non prima e non per timeout.
 - origine: PDF (stessa frase).
 - stato: **IMPLEMENTATO** — T9 rimuove la voce da T11, T5 riprende dalla fase in cui era.
- (c) RF02.1.1.2.3 - Il blocco vale anche per chi e' gia' fermo li'
- enunciato: un convoglio gia' fermo nella stazione che si guasta non riparte finche' il guasto e' aperto.
 - origine: **derivato** (altrimenti il guasto bloccherebbe solo chi arriva dopo, e la stazione non sarebbe davvero inagibile).
 - stato: **IMPLEMENTATO** — stesso meccanismo T11 / T5: la condizione e' sulla **prossima** tappa, che per un treno in sosta e' quella in cui si trova.

5.1.2 RF02.1.2 - Guasto del treno

1. RF02.1.2.1 - Il guasto viene memorizzato in centrale
 - enunciato: quando un convoglio si guasta, la centrale ne apre e persiste il guasto.
 - origine: PDF (§3.B: il treno segnala i guasti alla centrale).
 - stato: **IMPLEMENTATO** — sensore di bordo **POST /treno/sensore/guasto (T10)** -> alert MQTT (T9=/gateway) -> =M6 -> entita' **Guasto (M7)**.
2. RF02.1.2.2 - Gestione delle conseguenze
 - (a) RF02.1.2.2.1 - Treno guasto **in stazione** => stazione inagibile
 - enunciato: se il convoglio si guasta mentre e' fermo in stazione, quella stazione diventa **GUASTA**.
 - origine: PDF (§3.A: "un treno deragliato che inibisce l'uso di una certa stazione").
 - stato: **IMPLEMENTATO** — la decisione e' presa in **M6** sulla base della posizione del convoglio.

- dipende da: RF02.2.2.1 (sapere se il treno e' in stazione o in tratta).
- (b) RF02.1.2.2.2 - Treno guasto **in tratta** => tratta inagibile
- enunciato: se il convoglio si guasta fra due stazioni, e' la tratta a diventare inagibile.
 - origine: PDF (stessa frase, "o di una particolare linea della stazione").
 - stato: IMPLEMENTATO — stessa logica di M6.
 - **correzione:** la vecchia formulazione diceva "si vuole che la **stazione** tratta' risultera' inusabile", che confonde le due entita'. Qui la stazione non c'entra: il treno e' fermo in mezzo.

5.1.3 RF02.1.3 - Ciclo di vita del guasto

- enunciato: un guasto ha uno stato (aperto, presa in carico, chiuso), un'assegnazione e una traccia storica; non ci sono due guasti aperti sulla stessa sorgente per lo stesso motivo.
- origine: **derivato** (senza l'unicita' il watchdog aprirebbe un guasto ogni 10 secondi).
- stato: IMPLEMENTATO — M7; l'unicita' e' garantita da `TrafficLogicEngine.getGuastoApertoPerS` le tracce sono `StoricoGuasto` e `StoricoAssegnazioneGuasto`.

5.1.4 RF02.1.4 - Chiusura automatica quando la causa rientra

- enunciato: se un guasto era stato aperto dal watchdog per silenzio del nodo, il ritorno del battito lo chiude da solo, senza bisogno del tecnico.
- origine: **derivato** (un guasto dedotto va disfatto dalla stessa deduzione che l'ha creato, altrimenti la lista degli allarmi si riempie di falsi positivi che nessuno chiude).
- stato: IMPLEMENTATO — `IngestionService.onHeartbeat` (M4).

5.2 RF02.2 - Gestione dei treni

5.2.1 RF02.2.1 - Gestione dei ritardi

1. RF02.2.1.1 - Il ritardo si accumula quando il treno e' fermo e dovrebbe circolare

- origine: PDF (§3.C, Processing: “calcolo dei ritardi”).
 - stato: IMPLEMENTATO — T5: il contatore cresce mentre il convoglio e’ trattenuto.
- (a) RF02.2.1.1.1 - Ritardo per guasto
- stato: IMPLEMENTATO. Dipende da RF02.1.
- (b) RF02.2.1.1.2 - Ritardo per ordine dell’amministratore
- stato: FUORI AMBITO. Non e’ chiesto dal PDF e non l’ho implementato: non esiste un comando “rallenta questo treno”. Resta scritto qui perche’ era nella lista originale e per dire esplicitamente che e’ una scelta e non una dimenticanza.

5.2.2 RF02.2.2 - Servire le informazioni di stato del convoglio

1. RF02.2.2.1 - Posizione: su quale tratta o in quale stazione si trova
 - origine: PDF (§ Stato Interno del digital twin: stazione precedente, prossima stazione).
 - stato: IMPLEMENTATO — T5 (fase del viaggio) + T8 (telemetria ogni 5s) -> M3.
 - **nota:** questo e’ il **nodo a diamante** del grafo delle dipendenze. “Dove si trova il treno” e’ il predicato da cui dipendono la gestione dei guasti (stazione inagibile o tratta inagibile), la modifica della tratta e il watchdog dei treni fermi. Togliendolo cadono sei requisiti insieme.
2. RF02.2.2.2 - Ritardo accumulato
 - stato: IMPLEMENTATO. Dipende da RF02.2.1.
3. RF02.2.2.3 - Stato del convoglio
 - enunciato: il convoglio dichiara come sta (FERMO, IN_{VIAGGIO}, EMERGENZA, SOPPRESSO), che e’ cosa diversa da dove sta.
 - origine: **derivato** (servono due variabili ortogonali: un treno in emergenza resta nella fase di viaggio in cui era, e quando l’emergenza rientra riprende da li’).
 - stato: IMPLEMENTATO — T6 lato treno, normalizzato in M13 sulla colonna `Treni.stato`.

4. RF02.2.2.4 - Velocita' e passeggeri

- origine: **scelta** (rendono la simulazione leggibile sulla mappa).
- stato: IMPLEMENTATO — T8.

5.2.3 RF02.2.3 - Andata e ritorno sullo stesso itinerario

- enunciato: raggiunto il capolinea il convoglio inverte la marcia e rifa' l'itinerario al contrario, con sosta doppia al capolinea e tempo di percorrenza letto sulla tappa opposta.
- origine: PDF (§3.C: "il viaggio che ogni treno deve fare (elenco di stazioni) sia all'andata, che al ritorno"; e la direzione A/R compare anche nella richiesta della prossima stazione).
- stato: IMPLEMENTATO — T7.
- **nuovo**: mancava del tutto nella lista vecchia, pur essendo una richiesta esplicita.

5.2.4 RF02.2.4 - Passaggi asimmetrici in stazione

- enunciato: un convoglio puo' uscire da una stazione senza esserci entrato (stazione di partenza) e puo' entrare in una stazione senza uscirne (capolinea).
- origine: PDF (§3.A, testuale: "un treno puo' uscire da una stazione senza entrarvi [...] oppure entrare senza uscirvi").
- stato: IMPLEMENTATO — alla prima partenza `TrainJourneyEngine.avviaViaggio` non pubblica nessuna ENTRATA, solo la USCITA a fine sosta; al capolinea la USCITA arriva solo dopo l'inversione.
- **perche' conta**: e' il caso che rompe gli abbinamenti ingenui entrata/uscita nello storico dei transiti. Chi conta le coppie deve sapere che le prime e le ultime sono spaite per costruzione.

5.3 RF02.3 - Gestione delle stazioni

5.3.1 RF02.3.1 - Informazioni rese alla visualizzazione

- enunciato: la stazione sa dire quali treni ha sui binari, chi e' entrato e quando, chi e' uscito e quando.

- origine: PDF (§3.A, funzionalita' del nodo stazione).
- stato: IMPLEMENTATO.
- dove: S11 (presenza sui binari), esposto da GET /stazione/treni; il transito ufficiale verso la centrale nasce in S8 e viene ingerito da M5.
- dipende da: RF02.1.1.1.1 — la cache che serve la visualizzazione e' la **stessa** nata per sopravvivere alla disconnessione, non e' una seconda cache. Nella lista vecchia il legame non era scritto e sembravano due cose diverse.

5.3.2 RF02.3.2 - Heartbeat verso la centrale

- enunciato: la stazione manda un battito periodico alla centrale; il suo silenzio e' il segnale che la centrale usa per dedurre la caduta.
- origine: PDF (§3.A, Rilevamento: “si suggerisce un meccanismo di Heartbeat/Keepalive [...] sia tra Centrale e Stazioni”).
- stato: IMPLEMENTATO — HeartbeatGenerator ogni 10s su railway/station/{id}/heartbeat (S3), consumato da M4.
- **nuovo**: mancava nella lista vecchia, pur essendo la meta' del meccanismo di RF02.6.1.

5.3.3 RF02.3.3 - Keepalive dei sensori verso la stazione

- enunciato: i sensori di binario mandano un keepalive alla stazione; se uno tace, la stazione lo segnala come guasto alla centrale.
- origine: PDF (stessa frase: “sia tra Stazioni e relativi sensori. [...] se una Stazione non ricevesse Keep Alive da uno o piu' sensori, segnalerebbe lo stato di guasto alla Centrale”).
- stato: IMPLEMENTATO — POST /stazione/sensore/heartbeat (S6), sorveglianza in S7 (job da 10s), stato risultante in S10.
- **nuovo**: mancava, ed e' il livello di heartbeat piu' facile da dimenticare perche' non passa dal broker.

5.3.4 RF02.3.4 - Stato operativo dichiarato dalla stazione

- enunciato: la stazione dichiara di se' due soli stati, ONLINE e GUASTA.
- origine: derivato.
- stato: IMPLEMENTATO — S10.
- **nota:** sono due e non quattro come nella cache della centrale (M12). OFFLINE e MANUTENZIONE la stazione non li puo' conoscere di se' stessa: il primo e' una deduzione della centrale sul proprio silenzio (una stazione che si crede offline avrebbe comunque il canale per dirlo, e allora non lo e'), il secondo e' una decisione presa dal tecnico. Chi legge i due diagrammi in parallelo deve sapere che descrivono lo stesso oggetto da due punti di vista con informazione diversa.

5.4 RF02.4 - Validazione del nodo di campo

5.4.1 RF02.4.1 - L'ID dichiarato viene validato sul database centrale

- enunciato: un nodo (treno o stazione) che si avvia dichiara il proprio ID alla centrale; se l'ID esiste a database il nodo entra nel sistema, altrimenti no.
- origine: scelta (il PDF non lo chiede; l'ho aggiunto perche' senza, chiunque avvii un processo con un ID inventato inquina la centrale con telemetria di un convoglio che non esiste).
- stato: IMPLEMENTATO — richiesta e risposta su MQTT: T2 / S2 lato campo, ExistIdForEdge e ExistIdForEdgeStazione lato centrale (M2).

5.4.2 RF02.4.2 - Esito negativo: il processo non prosegue

- enunciato: se l'ID non e' riconosciuto il processo termina con codice di uscita 1, non resta acceso a vuoto.
- stato: IMPLEMENTATO — TrainDatabaseValidator / StationDatabaseValidator.

5.4.3 RF02.4.3 - Dipendenza fattorizzata

- Non e' un requisito a se': e' la regola di lettura del grafo. Senza RF02.4 nessun nodo entra nel sistema, quindi non arriva nessun dato e cadono per intero RF02.1, RF02.2 e RF02.3. Per questo nel grafo la freccia di

dipendenza e' disegnata una volta sola sui tre rami invece che su ogni foglia.

5.5 RF02.5 - Modifica della tratta a sistema acceso

Origine: PDF (§3.C: l'amministratore "puo' modificare le stazioni attraversate dai treni").

5.5.1 RF02.5.1 - Treno fuori dalla nuova tratta: viene fermato in attesa di soppressione

- **enunciato:** se dopo la modifica il convoglio non e' piu' assegnato ad alcun itinerario, smette di viaggiare e resta in attesa di essere soppresso.
- **stato:** IMPLEMENTATO — `updateTratta` sgancia i treni non piu' in elenco e manda comunque a tutti (vecchi e nuovi) l'evento `ITINERARIO_AGGIORNATO`; il convoglio sganciato ricarica, riceve 404 da `GET /api/treni/{id}/itinerario` e resta fermo riprovando ogni 15s (T3).
- **correzione:** la vecchia frase era "il treno **non** e' fuori dalla tratta il treno viene fermato in attesa di essere soppresso". Con quel "non" diceva l'opposto, ed era anche in contraddizione con il grafo delle dipendenze, dove la stessa voce era gia' scritta giusta.
- **nota di codice:** i treni da avvisare vanno letti **prima** di riassegnare le associazioni, altrimenti quelli appena sganciati non risultano piu' dalla query e continuano a girare sulla tratta vecchia senza saperlo. E' commentato in `RestApiGateway.updateTratta`.

5.5.2 RF02.5.2 - Treno dentro la nuova tratta: la segue dal punto in cui si trova

- **enunciato:** se il convoglio e' ancora assegnato, dovrebbe proseguire sul nuovo itinerario a partire dalla posizione in cui la modifica lo ha colto.
- **stato:** PARZIALE.
- **cosa funziona:** la notifica arriva (`ITINERARIO_AGGIORNATO` mirato sull'ID del convoglio, T9) e l'itinerario viene ricaricato al tick successivo (`TrainJourneyEngine.tick` alza il flag, `tentaCaricamentoItinerario` ricarica in REST).

- **cosa non funziona:** la ricarica azzerava `viaggioAvviato`, quindi `avviaViaggio()` riposiziona il convoglio su `indiceStazione = 0` con `direzione = "andata"`.
In pratica il treno **riparte dal primo capolinea del nuovo itinerario**, non da dove si trovava. Se la modifica lo coglie a meta' tratta, la posizione fa un salto.
- **come si chiuderebbe:** dopo la ricarica, cercare nel nuovo itinerario la stazione che il convoglio aveva appena lasciato (`stazioneCorrente`); se c'e', ripartire da quell'indice conservando direzione e ritardo; se non c'e' piu', ricadere sul comportamento attuale. Sono una decina di righe in `avviaViaggio`, ma preferisco dichiararlo qui piuttosto che scrivere che e' fatto.
- dipende da: RF02.2.2.1.

5.6 RF02.6 - Rilevazione automatica delle anomalie (watch-dog)

Enunciato del ramo: la centrale deduce da sola le anomalie che i nodi di campo non possono segnalare, prima fra tutte la propria caduta.

Origine: PDF (§3.A Fail-Stop: "se una stazione smette di inviare dati alla Centrale Operativa, quest'ultima non capirebbe se la stazione sia guasta o se i treni non transitano"; §3.C Alerting: "generazione automatica di allarmi in caso di anomalie, es. treno fermo tra due stazioni per troppo tempo").

Nuovo: tutto questo ramo mancava nella lista vecchia.

5.6.1 RF02.6.1 - Stazione muta oltre la soglia => OFFLINE e guasto automatico

- stato: IMPLEMENTATO — `FaultMonitor.controllaHeartbeat`, job ogni 10s, soglia `fault.heartbeat.timeout.secondi` (30s di default). Macchina M8.
- dipende da: RF02.3.2 (senza il battito non c'e' il silenzio da misurare).
- si chiude da solo per RF02.1.4.

5.6.2 RF02.6.2 - Treno fermo fuori stazione oltre la soglia => allarme automatico

- stato: IMPLEMENTATO — `FaultMonitor.controllaTreniFermi`, soglia `fault.treno.fermo.timeout.secondi` (90s di default).

- dipende da: RF02.2.2.1 — “fuori stazione” e’ un predicato sulla posizione, ed e’ il motivo per cui il nodo a diamante regge anche questo ramo.

5.6.3 RF02.6.3 - Snapshot periodico verso la dashboard

- enunciato: ogni 10 secondi la centrale spinge sul canale realtime una fotografia completa dello stato.
- origine: **derivato** (serve a riallineare chi si e’ appena collegato o ha perso messaggi; e’ il vero significato dei “10 secondi” che nella lista vecchia erano scritti come polling).
- stato: IMPLEMENTATO — `FaultMonitor.broadcastSnapshot` -> M11.

5.7 RF02.7 - Storicizzazione

- enunciato: oltre allo stato corrente, il sistema conserva la traccia di cio’ che e’ successo: transiti, cambi di stato dei treni e delle stazioni, guasti, assegnazioni degli operatori e itinerari percorsi.
- origine: PDF (§3.C Storage: stato attuale della linea e storico dei transiti).
- stato: IMPLEMENTATO — entita’ `StoricoTransito`, `StoricoStatoTreno`, `StoricoStatoStazione`, `StoricoGuasto`, `StoricoAssegnazioneGuasto`, `StoricoItinerario`.
- **scelta di progetto:** lo stato **runtime** dei nodi vive in cache (`TrafficLogicEngine`), non a database; a database ne resta la traccia storica. La cache si scrive a ogni frame di telemetria, la riga storica solo **al cambio di stato** (M3). E’ il motivo per cui un treno che manda 12 frame al minuto non genera 12 righe al minuto.
- e’ il lato dati di RF01.5.

6 RF03 - Ecosistema di agenti e comunicazione

Nuovo: ramo assente dalla lista vecchia, che parlava solo di dominio. Serve perche’ qui stanno le condizioni architetturali che il PDF valuta a parte (e a punti).

6.1 RF03.1 - Un processo per convoglio e un processo per stazione

- enunciato: ogni treno e ogni stazione sono processi indipendenti, non oggetti dentro un simulatore unico.
- origine: PDF (§ Simulazione Dinamica: “per rendere il progetto realistico (E OTTENERE LA SUFFICIENZA), la simulazione non deve essere un unico blocco di codice, ma un ecosistema di agenti”; inoltre la variante 7, che accorpa i servizi di monitoraggio delle stazioni in un unico servizio, costa 5 punti e **non** e’ stata presa).
- stato: IMPLEMENTATO — moduli **Treni** (:8082) e **Stazioni** (:8081), N istanze ciascuno, ID passato da argomento o property (T1, S1). I log per istanza stanno in **Stazioni/logs/**.

6.2 RF03.2 - MQTT dal campo alla centrale

- enunciato: la comunicazione fra nodi di campo e centrale passa dal broker, con topic gerarchici per entita’.
- origine: PDF (§4: MQTT dall’edge al core; la variante 4, stazioni che parlano REST, costa 5 punti e non e’ stata presa).
- stato: IMPLEMENTATO. Topic in uso:

topic	verso	contenuto
railway/train/{id}/validation	treno -> centrale	richiesta di validazione de
railway/train/{id}/validation-response	centrale -> treno	esito
railway/station/{id}/validation	staz. -> centrale	richiesta di validazione de
railway/station/{id}/validation-response	centrale -> staz.	esito
railway/train/{id}/telemetry	treno -> centrale	frame ogni 5s: posizione, s
railway/train/{id}/passaggio	treno -> tutti	ENTRATA / USCITA dic
railway/station/{id}/transit	staz. -> centrale	transito ufficiale rilevato c
railway/station/{id}/heartbeat	staz. -> centrale	battito ogni 10s
railway/alerts	condiviso	GUASTO, RESOLVED, S

6.3 RF03.3 - Un solo passaggio fisico, due strade indipendenti

- enunciato: lo stesso transito arriva alla centrale per due vie: dal sensore di terra (che scrive il transito storico) e dal convoglio (che aggiorna

posizione e prossima stazione). Le due vie non si sostituiscono a vicenda.

- origine: **derivato** dal modello del PDF, dove il treno “si presenta ai sensori” e il sensore “avverte la stazione, la quale inoltra il pacchetto dati alla Centrale”.
- stato: IMPLEMENTATO — `IngestionService.onTransit` e `onPassaggio` (M5).
- **nota**: e’ il punto in cui e’ piu’ facile fare confusione durante l’interrogazione. La domanda giusta e’ “perche’ due?”: perche’ il sensore e’ la fonte di verita’ per lo storico (esiste anche se il treno tace) e il convoglio e’ la fonte di verita’ per la posizione (la sa prima e meglio del sensore). Se ne togliessi una perderei o lo storico o il tempo reale.

6.4 RF03.4 - REST dai sensori al nodo di campo

- enunciato: i sensori non toccano il modello del nodo: parlano solo attraverso le API REST che il nodo espone.
- origine: **scelta** (decisione di progetto documentata nel contratto di integrazione; il PDF lascia liberta’ sul collegamento sensore-nodo).
- stato: IMPLEMENTATO.
 - stazione (:8081, StationIngestion, S6): POST /stazione/sensore/treno, /sensore/guasto, /sensore/heartbeat, /rete/offline, /rete/online; GET /stazione/info, /stato, /buffer, /treni;
 - treno (:8082, TrainIngestion, T10): POST /treno/sensore/guasto, /sensore/passaggio, /tratta; GET /treno/info, /stato, /viaggio, /tratta.
- **perche’ cosi’**: tenere i sensori fuori dal modello significa che la stessa macchina a stati vale sia con la console interattiva della demo sia con un chiamante esterno, senza percorsi di codice alternativi.

6.5 RF03.5 - Itinerario e prossima stazione serviti dalla centrale

- enunciato: il convoglio scarica dalla centrale l’itinerario completo all’avvio; in piu’ la centrale sa rispondere alla domanda “data la coppia treno + stazione attuale + direzione, qual e’ la prossima stazione?”.

- origine: PDF (§ Simulazione Dinamica: entrambe le opzioni, tabella precaricata in memoria **oppure** prossima stazione chiesta di volta in volta).
- stato: IMPLEMENTATO — GET /api/treni/{id}/itinerario (T3 -> M10) e GET /api/prossima-stazione. Il motore usa la tabella precaricata; l'endpoint puntuale resta disponibile per l'altra modalita'.
- **nota**: sono le due sole GET che il filtro di autorizzazione lascia aperte, perche' i processi di campo non fanno login (vedi RF04.2).

7 RF04 - Sicurezza degli accessi e dei canali

Nuovo: ramo assente dalla lista vecchia. Ci sono dentro punti di valutazione espliciti.

7.1 RF04.1 - Autenticazione tradizionale

- enunciato: gli utenti sono memorizzati sul server; l'accesso avviene con login e password e produce un token di sessione.
- origine: PDF variante 2 (esplicitamente ammessa al posto di OAuth2, **nessuna penalizzazione**).
- stato: IMPLEMENTATO — entita' `Utente`, `AuthController`, `SessioniAttive` (M9).

7.2 RF04.2 - Autorizzazione per ruolo

- enunciato: ogni chiamata sotto /api porta il token; le letture sono permesse a entrambi i ruoli, la CRUD sull'anagrafica al solo amministratore.
- origine: PDF (due tipologie di utenti con poteri diversi).
- stato: IMPLEMENTATO — `FiltroAutorizzazione`, filtro JAX-RS a priorita' di autenticazione.
- eccezioni dichiarate, ognuna con il suo motivo:
 - POST /api/auth/login pubblica: e' li' che si ottiene il token;

- le **OPTIONS** passano: la preflight del browser non porta header applicativi e senza questo il CORS fallisce prima della chiamata vera;
- le due GET di RF03.5 restano aperte: i processi Treno e Stazione non fanno login, la loro “autenticazione” e’ la validazione dell’ID di RF02.4. Sono due sole letture e nessuna scrittura.
- **nota onesta:** prima la distinzione fra i due ruoli era solo grafica, cioe’ il frontend nascondeva i pulsanti ma chiunque poteva cancellare un treno con un `curl`. Il filtro nasce per chiudere proprio quel buco, ed e’ il tipo di domanda che vale la pena aspettarsi.

7.3 RF04.3 - Canali cifrati

- enunciato: tutti i canali (MQTT verso il broker e HTTP verso la centrale) possono girare cifrati.
- origine: PDF variante 6 — **non** implementare TLS costa 5 punti, quindi implementarlo e’ un requisito da dichiarare.
- stato: **IMPLEMENTATO** come profilo — profilo Quarkus `%tls` su tutti e tre i moduli, MQTT su 8883 con truststore PEM, HTTPS della centrale su 8444; certificati generati da `BrokerMosquitto/tls/gen-certs.sh`. Si attiva con `-Dquarkus.profile=tls`.
- **attenzione:** sotto profilo TLS **ogni** canale deve avere `ssl=true`. Un solo canale dimenticato apre una socket in chiaro su un listener TLS: l’handshake fallisce e il sintomo che si vede e’ un nodo che non valida l’ID, non un errore di certificato. E’ il motivo per cui negli `application.properties` gli override `%tls.*` sono ripetuti canale per canale invece che raggruppati.

8 Grafo delle dipendenze aggiornato

Stesse convenzioni del grafo precedente (`00_grafo_dipendenze_requisiti.puml`): linea piena `*-` per la decomposizione, freccia tratteggiata `..>` per la dipendenza, cioe’ “togliendo il bersaglio il requisito non e’ piu’ soddisfacibile” (test di eliminazione). Il file `00_` descrive la lista vecchia ed e’ quindi superato da questo.

Per non riempire il disegno di frecce ripetute vale la regola gia’ usata: se tutte le parti di A dipendono da D, la freccia si mette una volta sola su A.

RF	requisito (in breve)	macchine	codice principale
RF01.1.1	mappa del traffico	M3, M11	TrafficMapPage.tsx, RealtimeWebSoc
RF01.1.2	elenco treni	T8, M3, M13	GET /api/treni, TrainsPage.tsx
RF01.1.3	elenco stazioni	S11, M12	GET /api/stazioni, StationsPage.ts
RF01.1.4	push realtime	M11	/ws/realtime, websocketClient.ts
RF01.1.5	KPI di sintesi	M10	TrafficLogicEngine.kpiDashboard
RF01.2	allarmi	M6, M7, M8	GET /api/allarmi, AlertsPage.tsx
RF01.3	CRUD anagrafica	M10	RestApiGateway, components/admin/*
RF01.4.1	invio manutenzione	M10, S9	POST /api/stazioni/{id}/manutenzi
RF01.4.2	risoluzione allarme	M7, M10, T9	POST /api/allarmi/{id}/risolvi
RF01.4.3	soppressione treno	M10, T9, T6	POST /api/treni/{id}/sopprimi
RF01.5	storico transiti	M5	GET /api/transiti, StoricoTransito
RF01.6	login / sessione	M9	AuthController, SessioniAttive
RF02.1.1.1	cache e reinvio	S4, S5, S7	LocalBuffer
RF02.1.1.2	stazione non percorribile	T9, T11, T5	TrainGateway.riceviAlert, TrainJou
RF02.1.2	guasto del treno e conseguenze	T10, M6, M7	TrainIngestion, IngestionService.c
RF02.1.4	chiusura automatica	M4	IngestionService.onHeartbeat
RF02.2.1	ritardi	T5	TrainJourneyEngine
RF02.2.2	posizione e stato	T5, T6, T8	TrainDB, TrainElab
RF02.2.3	andata e ritorno	T7	TrainJourneyEngine (inversione al cap
RF02.2.4	passaggi asimmetrici	T7, M5	TrainJourneyEngine.avviaViaggio
RF02.3.1	info della stazione	S8, S11	StationGateway, GET /stazione/tren
RF02.3.2	heartbeat stazione	S3, M4	HeartbeatGenerator
RF02.3.3	keepalive sensori	S6, S7, S10	POST /stazione/sensore/heartbeat,
RF02.4	validazione del nodo	T2, S2, M2	TrainDatabaseValidator, ExistIdFor
RF02.5	modifica della tratta	M10, T9, T3	RestApiGateway.updateTratta
RF02.6	watchdog	M8	FaultMonitor
RF02.7	storicizzazione	M3, M5, M7	entita' Storico*
RF03.1	ecosistema di agenti	T1, S1, M1	moduli Treni e Stazioni, N istanze
RF03.2	topic MQTT	tutte	application.properties dei tre mod
RF03.4	REST dei sensori	S6, T10	StationIngestion, TrainIngestion
RF03.5	itinerario e prossima stazione	T3, M10	GET /api/treni/{id}/itinerario, /a
RF04.2	autorizzazione per ruolo	M9	FiltroAutorizzazione
RF04.3	TLS	-	profilo %tls, BrokerMosquitto/tls/ge

10 Riepilogo di cio' che e' dichiarato non fatto

Tenerlo in un elenco unico e corto conviene: sono le cose su cui vale la pena avere una risposta pronta invece di scoprirle davanti al prof.

requisito	stato	perche'
RF02.5.2	PARZIALE	dopo la modifica della tratta il convoglio ricarica ma
RF02.2.1.1.2	FUORI AMBITO	non esiste un comando “rallenta questo treno”: non
previsione dell’orario di arrivo	FUORI AMBITO	il PDF cita “previsioni di arrivo” accanto al calcolo
vie multiple in stazione	FUORI AMBITO	il PDF la mette come opzione (“se volete gestire and

11 Nota sui file vicini

- `00_grafo_dipendenze_requisiti.puml` disegna la lista **vecchia**: e’ superato dal grafo di questo file e va tenuto solo come storia della revisione, oppure riallineato.
- La sezione “Elenco maggiomente formale dei requisiti” della documentazione finale puo’ essere sostituita da un rimando a questo file, cosi’ non ci sono due liste che dicono cose diverse.